DAY 34

Ensemble Learning

Bagging • Boosting • Stacking • Random Forest • Evaluation

60-Day Data Science Learning Journey

A beginner-friendly practical guide with original diagrams, Python examples, evaluation metrics, exercises, and solutions.

1. Learning objectives

• Define ensemble learning and explain why it can help.

• Distinguish Bagging, Boosting, and Stacking.

• Understand bootstrap sampling, Random Forest, and OOB scoring.

• Train and compare ensemble classifiers using scikit-learn.

• Use accuracy, precision, recall, F1-score, and cross-validation responsibly.

2. What is Ensemble Learning?

Ensemble learning combines predictions from multiple models into one final prediction. It may improve stability or predictive performance, but improvement is not guaranteed. **Roman Urdu:** Multiple models ki predictions combine karke aik final prediction hasil karna ensemble learning kehlata hai.

Majority voting example

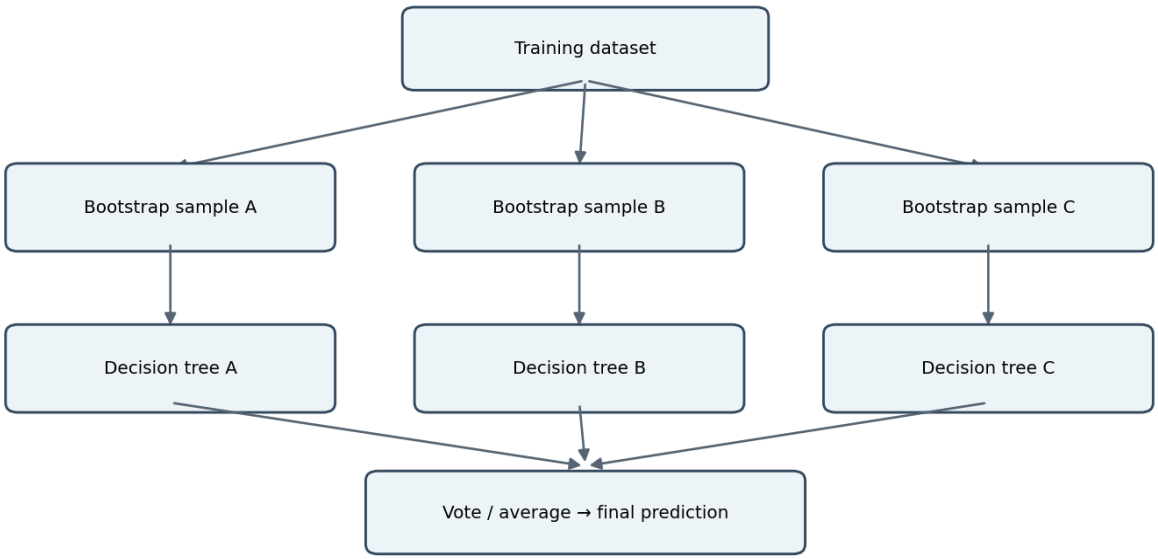
Model	Prediction
1	Pass
2	Pass
3	Fail
4	Pass
5	Pass

Four Pass votes and one Fail vote produce Pass by majority voting.

3. Bagging

Bagging means Bootstrap Aggregating. It trains estimators on bootstrap samples drawn with replacement, then combines predictions. A sample may occur multiple times in one bootstrap sample or be left out of it. Bagging often helps reduce variance for unstable estimators.

Bagging: fit models on bootstrap samples, then aggregate



Classification usually combines predictions by voting. Regression commonly averages predictions.

4. Random Forest

Random Forest is a bagging-based ensemble of decision trees with randomized feature selection at splits. This randomness can reduce correlation among trees.

• **n_estimators**: number of trees.

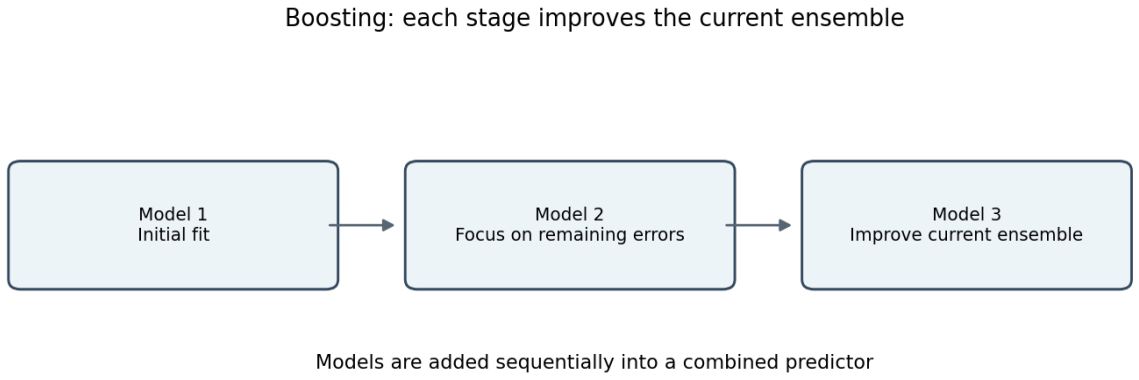
• **max_depth**: optional maximum tree depth.

• **max_features**: features considered at a split.

• **n_jobs=-1**: use all available CPU cores for supported parallel work.

5. Boosting

Boosting adds models sequentially. Each stage tries to improve the current ensemble. AdaBoost changes emphasis on difficult examples, while Gradient Boosting fits learners to reduce a loss function's remaining errors or negative gradients.



• **n_estimators**: number of boosting stages.

• **learning_rate**: scales each stage's contribution and interacts with the number of stages.

Examples include AdaBoost, Gradient Boosting, XGBoost, and LightGBM. XGBoost and LightGBM require separate installation.

6. Stacking

Stacking passes predictions from base models to a meta-model that learns how to combine them. To avoid leakage, the meta-model should be trained on out-of-fold predictions, not predictions from base models on the same examples used to fit them. scikit-learn's StackingClassifier uses cross-validation for this by default.

7. Compare ensemble methods

Method	Training	Combination	Example
Bagging	Bootstrap samples, often parallel	Vote / average	BaggingClassifier
Random Forest	Bagging + feature randomness	Vote / average	RandomForestClassifier
Boosting	Sequential stages	Additive / weighted	GradientBoostingClassifier
Stacking	Base models + meta-model	Learned combination	StackingClassifier

8. Evaluation metrics

• **Accuracy**: correct predictions divided by all predictions.

• **Precision**: among predicted positives, the fraction that are true positives.

• **Recall**: among actual positives, the fraction correctly found.

• **F1-score**: harmonic mean of precision and recall.

Accuracy can be misleading on imbalanced data. Choose metrics based on the consequences of false positives and false negatives, and check which class is treated as positive.

9. Out-of-Bag (OOB) evaluation

Bootstrap sampling leaves some training observations out of each tree's sample. These observations are out-of-bag for that tree. With oob_score=True and bootstrap sampling enabled, RandomForestClassifier can aggregate OOB predictions into a useful estimate. Still retain a separate test set for final evaluation.

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(
    n_estimators=200, oob_score=True,
    random_state=42, n_jobs=-1
)
rf.fit(X_train, y_train)
print(rf.oob_score_)
```

10. Complete Python workflow

The accompanying Python file contains a full runnable script with metrics, OOB score, and cross-validation. The dataset below is provided by scikit-learn for educational classification practice only, not medical diagnosis.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Bagging": BaggingClassifier(
        estimator=DecisionTreeClassifier(), n_estimators=100,
        random_state=42, n_jobs=-1),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, random_state=42, n_jobs=-1),
    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=100, learning_rate=0.1, random_state=42)
}
for name, model in models.items():
    model.fit(X_train, y_train)
    print(name, accuracy_score(y_test, model.predict(X_test)))
```

Code walkthrough

• X contains features and y contains labels.

• test_size=0.2 holds out 20% for testing.

• random_state supports reproducibility; stratify preserves approximate class proportions.

• Fit models on training data, then compare predictions on the same held-out test set.

• For careful tuning, use cross-validation on training data and reserve the test set for final evaluation.

11. Cross-validation and leakage prevention

Use cross-validation on training data to compare settings. Keep the final test set untouched during tuning. Learned preprocessing and feature selection belong inside a Pipeline so each fold fits transformations only on its training portion. Stacking should use out-of-fold predictions to train the meta-model.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(
    RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1),
    X_train, y_train, cv=cv, scoring="accuracy"
)
print(scores)
print("Mean CV score:", scores.mean())
```

12. Common mistakes

• Assuming ensembles always outperform a single model.

• Selecting models by training accuracy instead of unseen-data performance.

• Repeatedly tuning against the final test set.

• Ignoring class imbalance or reporting accuracy alone.

• Training a stacking meta-model on in-sample predictions.

• Treating an educational medical dataset as a diagnostic tool.

13. Practice exercises with solutions

Exercise 1: Votes are Pass, Pass, Fail, Pass, Fail. What does majority voting predict?

Solution: Pass, with three votes versus two.

Exercise 2: Explain Bagging versus Boosting.

Solution: Bagging fits estimators on bootstrap samples and aggregates them. Boosting adds estimators sequentially to improve the current ensemble.

Exercise 3: What does n_estimators=100 mean in Random Forest?

Solution: The forest contains 100 trees.

Exercise 4: Why not tune repeatedly on the final test set?

Solution: This leaks test-set information into model selection and makes the final score less trustworthy.

Exercise 5: Why use precision, recall, and F1-score?

Solution: They reveal different kinds of classification errors and can be more informative than accuracy on imbalanced data.

Exercise 6: What is an OOB observation?

Solution: A training row left out of a particular estimator's bootstrap sample.

14. Revision checklist

• ■ I can define ensemble learning.

• ■ I can distinguish Bagging, Random Forest, Boosting, and Stacking.

• ■ I understand bootstrap samples and OOB scoring.

• ■ I can train and compare ensemble models.

• ■ I can explain accuracy, precision, recall, and F1-score.

• ■ I understand cross-validation and test-set separation.

Day 34 takeaway: Combining models is a tool, not a guarantee. Careful validation and appropriate metrics are essential.

GitHub repository: https://github.com/DeveloperDesigner-star/Datasciencepdfs